First Examination (110 pts ...

1. **(10 pts) Permutation generator**
Assume it takes O(1) to swap two elements (i.e., swap($a[i]$, $a[j]$).

```
Lines  void Permutations (char *a, const int k, const int m)
1        {// Generate all the permutations of a[k], ..., a[m]
2          if (k == m) { // output permutation
3              for (int i = 0; i <= m; i++)  cout << a[i] << " ";
4              cout << endl;
5          }
6          else{ // a[k:m] has more than one permutation. Generate these recursively.
7              for (i = k; i <= m; i++)) {
8                  swap(a[k], a[i]);
9                  Permutations(a, k+1, m);
10                 swap(a[k], a[i]);
11             }
12         }
13       }
```

a. **(5 pts)** Please insert a new variable, *count*, into the program and show a simplified version of the original program and show the time complexity according to it.

b. **(5 pts)** Please build a table which lists the total number of steps contributed by each statement as follows and show the time complexity according to it.

line	s/e	frequency	total steps
1			
2			
⋮			
13			
		Total number of steps	

2. **(10 pts)** The class Bag is a container which is stored by an integer array. Please modify the following code using the Template.

```
class Bag {
public:
  Bag(int bagCapacity = 10);
  ~Bag();

void Pop();

private:
  int *array;
  int capacity; // capacity of array
  // array position of top element
  int top;
};
```

```
Bag::Bag(int bagCapacity):
  capacity (bagCapacity) {
  if (capacity < 1) throw "Capacity must be > 0";
  array = new int [capacity];
   top = -1;
}

Bag:: ~Bag() { delete [] array; }

void Bag::Pop() {
  if (IsEmpty()) throw "Bag empty! cannot delete";
  int deletePos = top/2;
  copy(array+deletePos+1, array+top+1, array+deletePos); //
        compact array
   top--;
```

3. 問答題 (30 pts)

a. (3 **pts**)When you estimate the time complexity of an algorithm, what constitutes a step is defined after which property has been determined?

b. (3 **pts**) In addition to 1) support of objects and 2) requirement of objects belong to a class, what is the third property to constitute an object-oriented language?

c. (3 **pts**) Please show the location when you define class ChainIterator with respect to the class Chain which is singly linked list?

d. (3 **pts**) How many space is required when a representation 1 and a representation 2 are used if we have a polynomial equation $f(x)=3x^{100}+4x^{98}+50x^{30}+49$?

Representation 1

```
private:
   int degree;
   float *coef;


Polynomial::Polynomial(int d)
{
      degree = d;
      coef = new float [degree+1];
}
```

Representation 2

```
class Polynomial;

class Term {
friend Polynomial;
private:
   float coef; // coefficient
   int exp; // exponent
};

// The private data members of Polynomial
private:
   Term *termArray; // array of nonzero terms
   int capacity; // size of termArray
   int terms; // # of nonzero terms
```

e. (3 **pts**) Given a spare matrix which is represented by a circular list, how many nodes do you have if it contains r nonzero elements?

f. (3 **pts**) Please show the equation to represent the address of a C++ array $a[i_1][i_2]\ldots[i_n]$ if it starts at the address α (note that it is stored in the **row major**) and the dimension of the array is $a[u_1][u_2]\ldots[u_n]$.

g. (3 **pts**) What are the nested classes? Compared to decomposed classes, what is the problem with the nested classes?

h. (3 **pts**) Given a circular queue which is stored by an array, where front and rear are the pointers for insertion and deletion of an element from the array, please complete the following function:

```
class Queue {
public:
   Queue(int queueCapacity = 10);
   ~Queue();
   void Push (const T& item);
   void Pop();
private:
   T *queue;
   int front; // one counterclockwise from front
   int rear, capacity;
};
```

```
template <class T>
void Queue<T>::Push(const T& x)
{
   if (_____) {
      // double capacity right before full
      // code to double capacity comes here
   }

   queue[rear] = x;
}
```

i. (3 **pts**) Compared to the singly linked list, what is the most impo
of the doubly linked list?

j. (3 **pts**) Please explain the purpose of setting the available list *av* as static data member of the class CircularList as follows:

```
template <class T>
class CircularList {
   static ChainNode <T> *av; // available space list
public:
   // public members here
private:
   // other private members here
};
```

4. (10 **pts**) Give a table which characterizes the priority of the operators.

The expression *e* is A && B ||C||!(E > F)

ABS & & C||E<F>!||

Note that you have to show the process of change in a stack when you reply the following questions.

a. (2 **pts**) Show the equivalent **postfix** expression if we use a stack to evaluate the expression.

b. (5 **pts**) Show the process to evaluate the value of e according to the result obtained in (a) where A=True, B=True, C= False, E = 3, F=4.

c. (3 **pts**) Show the equivalent **prefix** expression if we use a stack to evaluate the expression. **Hint:** you have to scan *e* from right to left.

||||>FE<&&BA

!>EF

||&&AB!C!>EF

priority	ISP	ICP
(		0
!	1	1
* / %	2	2
+ -	3	3
<, >	4	4
==, !=	5	5
&&	6	6
||	7	7
(	8	

```
void Postfix(Expression e) {
Stack<Token> stack; // initialize stack
   stack.Push('#');
   for (Token x = NextToken(e); x != '#'; x=NextToken(e))
      if (x is an operand) cout << x;
      else if ( x == ')') { // unstack until '('
         for (; stack.Top() != '('; stack.Pop())
            cout << stack.Top();
         stack.Pop(); // unstack '('
      } else { // x is an operator
         for (; isp(stack.Top()) <= icp(x); stack.Pop())
            cout << stack.Top();
         stack.Push(x)
      }
// end of expression; empty stack
   for (; !stack.IsEmpty(); cout << stack.Top(), stack.Pop());
   cout << endl;
}
```

Department　Year　Class　Score

5. (30 pts) Please point the error and correct it.

a. Functional decomposition is addressed before a system has been decomposed into objects in the object-orientated decomposition.

b. Testing and debugging a program can become more easier because of data encapsulation.

c. Data encapsulation is to separate the specification of a data object and its implementation.

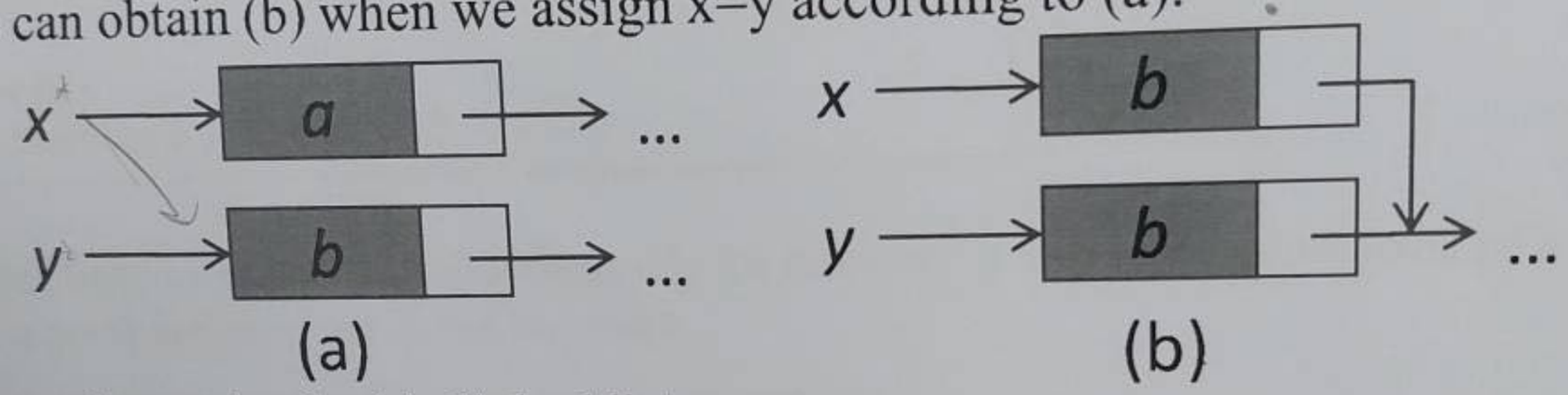
d. According to the time complexities $O(f_1)$ and $O(f_2)$, we can say the f_1 is faster than f_2, where $f_1=4n^3+n^2$ and $f_2=n^3+1$.

e. The space of a pointer of a data member as well as the associated data object cannot be deleted if the destructor of a user specified class is not defined.

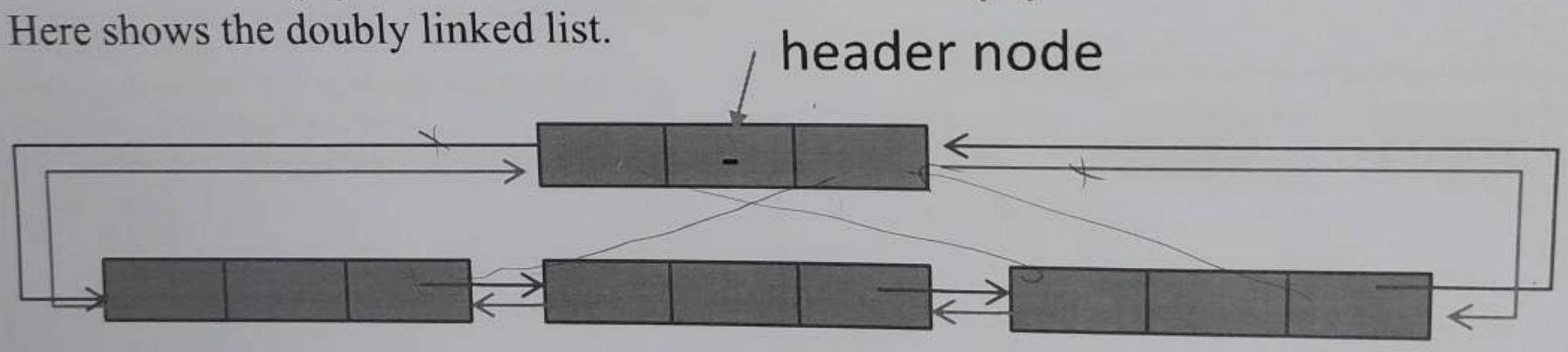
f. A derived class inherits all the non-private members data and functions of the base class including constructor and destructor.

g. If class B inherits from class A, then B IS-A A. This implies that A is more specific than B.

h. We can obtain (b) when we assign x=y according to (a).



i. Here shows the doubly linked list.



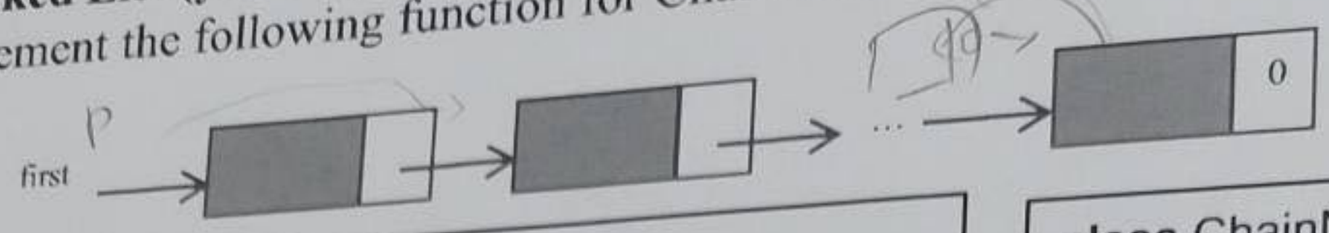
j. Please find the error and correct it in the following pseudo code.

```
class ChainIterator {
public:
    T* operator ->() const {return current->data;}

  // equality testing
    bool operator!=(const ChainIterator right) const
        {return current != right.current;}
    bool operator==(const ChainIterator right) const
        {return current == right.current;}
private:
    ChainNode <T>* current; };
```

```
class ChainNode {
    friend class Chain <T>;
    private:
        T data;
        ChainNode <T> *link;
};

class Chain { // definition here
public:
    Chain() {first = 0;}
private:
    ChainNode <T> *first;
};
```

6. (20 pts) Linked List (you only need to fill the answer in [])

a. Please implement the following function for ChainIterator which an iterator of a singly linked list.

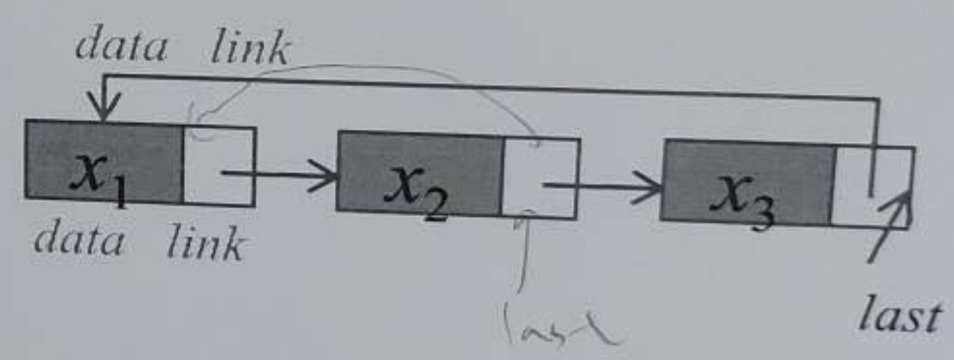


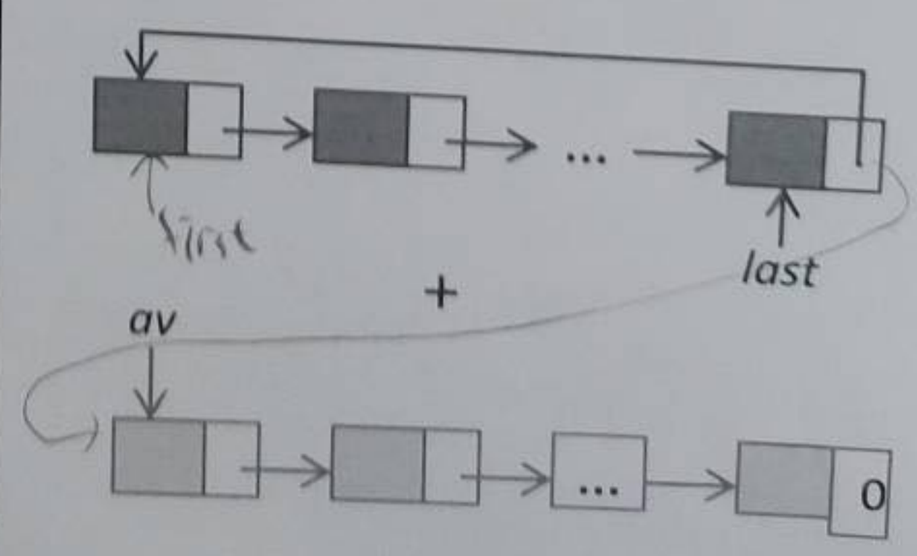

```
class ChainIterator {
public:
  ChainIterator& operator --(int ) { //postdecrement
    if ( [A] ) return 0;
    else {
      ChainNode* p = first;
      ChainIterator old;
      while ( [B] )
            [C];
      }
      [D];
      [E];
      return old;
    }
  }
}
private:
  ChainNode <T>* current; };
```

```
class ChainNode {
  private:
    int data;
    ChainNode * link;
}
```

b. (6 **pts**) Please complete the following code for inserting a new node to the back of a circle list with a point *last* pointing to the last node.

```
template <class T>
void CircularList<T>::InsertBack(const T& e)
{ // Insert e at the back of *this
  ChainNode <T> *newNode = new ChainNode <T>(e);
  if (last) {
    [F];
    [G];
    [H];

  } else {
    last = newNode;
    newNode->link = newNode;}
}
```



c. (4 **pts**) Please complete the following code for restoring nodes in a circuit list to an available list.

```
template <class T>
void CircularList<T>::~CircularList()
{ // Delete the whole circular list
  if (last) { // nonempty list
    ChainNode<T>* first = last->link;
    [I];
    [J];
    last = 0;
  }
}
```